Pipeline de Dados (Data Pipeline)

O foco total está no **dado**. É uma sequência de processos que extrai informações de uma ou mais fontes, transforma esses dados e os entrega em um destino.

- **O que move:** Dados brutos e processados (arquivos, registros de banco de dados, logs).
- **Objetivo:** Garantir que o dado chegue íntegro, limpo e rápido ao destino final (como um Data Warehouse).
- **Características:** Geralmente automatizado de ponta a ponta, lida com grandes volumes (Big Data) e opera em lote (batch) ou tempo real (streaming).
- **Exemplo clássico:** Processo de **ETL** (Extração, Transformação e Carga). Pegar dados de vendas do aplicativo, converter as moedas para Real e salvar no banco de relatórios da empresa.

📋 Workflow (Fluxo de Trabalho)

O foco total está na **tarefa e na lógica de execução**. É uma sequência de passos (manuais ou automatizados) necessários para concluir um processo de negócio ou de sistema.

- **O que move:** Status de tarefas, decisões e regras de negócio.
- **Objetivo:** Coordenar a ordem de execução, dependências e caminhos lógicos de um processo.
- **Características:** Inclui tomadas de decisão complexas (se o passo A falhar, faça o B; se tiver sucesso, faça o C) e pode envolver interações humanas (como uma aprovação de gerente).
- **Exemplo clássico:** Processo de **integração de novos funcionários (Onboarding)**. Criar o e-mail no sistema ➡️ Enviar contrato para assinatura humana ➡️ Liberar acessos apenas *após* a assinatura.

⚖️ Comparação Direta

Característica	Pipeline de Dados	Workflow
Elemento Principal	O **Dado** em si.	A **Tarefa** ou ação.
Lógica	Linear (origem ➡️ transformação ➡️ destino).	Ramificada (condicionais, loops, validações).
Interação Humana	Raramente possui (100% automatizado).	Comum (aprovações, revisões manuais).
Falhas	Foca em reprocessar o dado ou isolar registros ruins.	Foca em alertas, caminhos alternativos e travas de segurança.
Ferramentas Comuns	Apache Spark, dbt, AWS Glue, Fivetran.	Apache Airflow, Camunda, Zapier, Make.

> 💡 **Analogia simples:** Pense no **Pipeline de Dados** como uma tubulação de água (o foco é transportar a água da represa até a sua torneira filtrando as impurezas). O **Workflow** é o manual de operação da estação de tratamento inteira (se o nível do tanque subir, feche a válvula X; às 8h da manhã, envie um técnico para vistoria; se faltar energia, ligue o gerador).

Você está planejando a arquitetura de um projeto? Se quiser, pode me dizer se o seu desafio envolve **mover grandes volumes de bancos de dados** ou **orquestrar etapas de um sistema**, e eu te ajudo a escolher a melhor abordagem ou ferramenta (como Airflow, Prefect ou dbt).

o que é o stage area no pipeline de dados?

O staging area (ou área de estágio) é uma zona de armazenamento intermediária em um pipeline de dados onde os dados brutos são depositados temporariamente

antes de serem transformados e carregados no destino final (como um Data Warehouse).

Ela funciona como uma "sala de triagem" ou um "amortecedor" entre as fontes de origem e o sistema analítico principal.

🧱 Por que ela é necessária? (As 4 principais funções)

- **Redução de impacto na origem:** Extrair dados de sistemas de produção (como o banco de dados do seu e-commerce) pode deixá-los lentos. O pipeline copia os dados brutos o mais rápido possível para o *staging area*, liberando o sistema original imediatamente.
- **Isolamento e Segurança:** Realizar transformações complexas diretamente nos sistemas de origem é arriscado. Na *staging area*, os engenheiros de dados podem manipular, limpar e testar os dados sem qualquer chance de corromper o banco de dados oficial da empresa.
- **Consolidação de Fontes:** Se você precisa cruzar dados que vêm do Google Analytics, do Salesforce e de um banco MySQL, a *staging area* serve como o ponto de encontro onde todos esses formatos diferentes são guardados juntos antes de serem unificados.
- **Ponto de Recuperação:** Se o processo de transformação falhar no meio do caminho, você não precisa extrair tudo de novo da origem (o que gastaria tempo e processamento). Os dados já estão guardados em segurança na área de estágio prontos para uma nova tentativa.

🔄 Onde ela fica no processo?

No modelo tradicional **ETL** (Extract, Transform, Load), a *staging area* fica exatamente entre a Extração e a Transformação:

```
None
[Fontes de Origem] ➡️ (Extração) ➡️ [ STAGING AREA ] ➡️ (Transformação) ➡️ [Data Warehouse]
```

💻 Como ela é implementada na prática?

Dependendo da tecnologia da empresa, a *staging area* pode ser:

- **Pastas (Buckets) em Cloud Storage:** Como o AWS S3, Google Cloud Storage ou Azure Blob Storage (muito comum em arquiteturas modernas de *Data Lake*).
- **Esquemas temporários em Bancos de Dados:** Tabelas específicas dentro do próprio Data Warehouse (como tabelas `stg_` no Snowflake, BigQuery ou PostgreSQL) que são limpas após o processo acabar.

> 💡 **Analogia simples:** Imagine que você está mudando de casa. Em vez de tirar os móveis do caminhão e tentar decidir onde colocar cada um deles e montá-los direto nos quartos (o que geraria caos e trânsito na porta), você descarrega tudo rapidamente na **garagem** (sua *staging area*). Com tudo na garagem, você limpa os móveis, joga fora o que quebrou, organiza as caixas e depois leva tudo pronto para os cômodos definitivos.

Você está desenhando um pipeline usando alguma tecnologia específica? Me avise se quiser saber as **melhores práticas de nomenclatura** para tabelas de staging ou como estruturar essa área usando ferramentas como **dbt (data build tool)** ou **AWS S3**.

o que é ' Idempotência'?

Idempotência é a propriedade de uma operação que permite que ela seja executada múltiplas vezes sem alterar o resultado além da primeira aplicação. Em termos simples: não importa se você executa uma ação uma, dez ou cem vezes, o estado final do sistema será exatamente o mesmo da primeira execução.

É um conceito fundamental em matemática, mas se tornou um pilar crucial na **computação, APIs, arquitetura de software e pipelines de dados** para garantir a consistência dos sistemas.

💡 Exemplos Práticos do Dia a Dia

- **Botão de Elevador (Idempotente):** Se você aperta o botão para o 5º andar uma vez, o elevador vai para o 5º andar. Se você apertar o botão 20 vezes seguidas, ele continuará indo para o mesmo 5º andar. O resultado não muda.
- **Inserção de Cartão de Memória (Idempotente):** Empurrar um cartão já inserido no aparelho não faz nada novo. O estado final ("cartão inserido") permanece inalterado.
- **Adicionar um item ao carrinho (Não Idempotente):** Se você clicar em "Adicionar ao carrinho" cinco vezes, você terá cinco unidades do produto. O resultado acumula a cada clique.

💻 Por que ela é vital na Computação e Engenharia de Dados?

Sistemas de computação falham o tempo todo (quedas de rede, timeouts, servidores que reiniciam). Quando uma falha acontece, a reação padrão de um sistema automatizado é **tentar novamente (retry)**. É aí que a idempotência salva o dia:

1. Em APIs e Pagamentos (Prevenção de cobrança dupla)

Imagine que você compra um produto online. O banco processa o pagamento com sucesso, mas a rede oscila antes de enviar a confirmação para a tela do seu celular. O aplicativo tenta enviar a requisição de pagamento de novo.

- **Se o sistema for idempotente:** Ele reconhece (através de uma chave única) que aquele pagamento já foi feito e apenas responde "Sucesso", sem cobrar seu cartão novamente.
- **Se não for:** Você será cobrado duas vezes.

2. Em Pipelines de Dados e Staging Areas

Conectando com o assunto anterior, se o seu pipeline cair no meio da carga de dados da *staging area* para o *Data Warehouse* e precisar ser reiniciado:

- **Pipeline Idempotente:** Ele vai reprocessar os dados daquele dia, sobrescrevendo ou ignorando o que já existe, garantindo que nenhum registro fique duplicado.
- **Pipeline Não Idempotente:** Ele vai inserir os mesmos dados novamente, gerando relatórios duplicados e análises de negócios totalmente erradas.

3. Em Métodos HTTP

Na web, os métodos HTTP possuem regras estritas sobre isso:

- `GET`, `PUT`, `DELETE` ➡️ **São Idempotentes:** Deletar um usuário ID 40 dez vezes resulta na mesma coisa (o usuário não existe). Atualizar o e-mail do usuário para `joao@email.com` (`PUT`) várias vezes deixa o e-mail igual.
- `POST` ➡️ **Não é Idempotente:** Enviar um formulário de "Criar Usuário" cinco vezes vai criar cinco usuários diferentes no banco de dados (a menos que haja uma trava de idempotência por trás).

🛠️ Como se implementa a Idempotência?

A forma mais comum em software é através de uma **Chave de Idempotência (Idempotency Key)**. O cliente gera um código único (como um UUID) para aquela operação e o envia junto com a requisição. O servidor guarda essa chave por um tempo. Se receber outra requisição com a mesma chave, ele não executa a lógica de novo; apenas devolve a resposta que já estava guardada.

Ficou claro como esse conceito se aplica na prática? Se quiser, posso te mostrar um **exemplo de query SQL idempotente** (usando `MERGE` ou `UPSERT`) para pipelines de dados, ou como estruturar uma **chave de idempotência em uma API**. O que prefere ver?

> os comandos dbt para trnasofmração em dados tabulares podem ser realizados com dados em .parquet pelo pandas com a mesma eficiencia?

Não com a mesma eficiência. Embora ambos consigam realizar as mesmas transformações lógicas, o **dbt** (rodando em um Data Warehouse moderno) e o

Pandas (processando arquivos `.parquet`) operam sob arquiteturas e filosofias de computação completamente diferentes.

A principal diferença resume-se a onde o processamento acontece: **Memória RAM vs. Computação Distribuída/Bancos de Dados.**

🏛️ 1. O dbt e a Arquitetura Pushdown (Elástica)

O dbt não processa dados por conta própria. Ele funciona traduzindo seu código (SQL ou Python do dbt) em comandos nativos e enviando-os (*pushdown*) para o seu Data Warehouse (como **Snowflake, BigQuery, Databricks** ou mesmo um motor local como o **DuckDB**).

- **Escalabilidade:** Praticamente infinita. Se você precisar transformar 1 Terabyte de dados (sejam tabelas ou arquivos `.parquet`), o Data Warehouse distribui essa carga entre dezenas de máquinas na nuvem.
- **Eficiência em Consultas:** Motores analíticos modernos são colunares. Se o seu modelo dbt pede apenas 3 colunas de uma tabela com 200 colunas, o banco lê apenas o estritamente necessário do disco, de forma ultra otimizada.

🐼 2. O Pandas e a Arquitetura em Memória (In-Memory)

O Pandas foi desenhado para processar dados carregando-os diretamente na memória **RAM** do seu computador ou servidor local.

- **O Gargalo da RAM:** Para ler e transformar um arquivo `.parquet` de 10 GB com eficiência, o Pandas pode precisar de 30 GB a 50 GB de memória RAM disponível (já que ele descompacta e indexa os dados na memória). Se os dados excederem a memória, o script simplesmente vai quebrar com um erro de *Out of Memory (OOM)*.
- **Single-core por padrão:** O Pandas realiza a maioria das operações em apenas um núcleo do processador, não aproveitando todo o poder de máquinas modernas multi-core de forma nativa.

⚖️ Comparação Direta de Cenários

Para entender qual abordagem é mais eficiente para o seu caso, veja o comparativo:

Cenário / Volume de Dados	dbt (com Data Warehouse / DuckDB)	Pandas (com arquivos .parquet)
Volumes pequenos (< 2 GB)	Excelente, mas pode ser um exagero de infraestrutura se for um processo isolado.	**Ultra eficiente e rápido.** Ideal para análises rápidas e protótipos locais.
Volumes médios (2 GB a 50 GB)	Altamente eficiente, executa em segundos ou poucos minutos de forma previsível.	Começa a ficar lento, exige servidores com muita RAM e corre risco de travar.
Big Data (> 100 GB / Terabytes)	**Extremamente eficiente.** Escala horizontalmente na nuvem de forma automática.	**Inviável.** O Pandas não conseguirá carregar os dados na memória.
Governança e Linhagem	Nativo (gera diagramas de dependência, testes de dados e documentação).	Manual (você precisa controlar a ordem de execução dos scripts na mão).

🛠️ E se você quiser o melhor dos dois mundos localmente?

Se você gosta da velocidade de ler arquivos `.parquet` locais, mas quer a organização do dbt sem sofrer com as limitações de memória do Pandas, a recomendação moderna é usar o **dbt aliado ao DuckDB** (através do plugin `dbt-duckdb`).

O DuckDB lê arquivos `.parquet` diretamente na velocidade da luz, processa códigos SQL idênticos aos de grandes Data Warehouses e possui gerenciamento inteligente de memória que não quebra o computador como o Pandas.

Se você precisa obrigatoriamente usar Python para transformar os `.parquet`, frameworks como **Polars** ou **PySpark** são alternativas muito mais eficientes e escaláveis que o Pandas.

O seu objetivo atual é construir um **pipeline de dados local** rodando em sua máquina ou você está avaliando a **arquitetura de nuvem** de um projeto da empresa? Posso te sugerir como estruturar um projeto com **dbt-duckdb** se quiser rodar transformações rápidas em arquivos `.parquet`.

> Bom, em meu esquema medalion os dados são processados em silver. Nessa camada poderia então, já que recebera da camada raw os campos necessários usar o dbt em vez do pandas para transformar e promover para gold ou não?

Sim, com certeza. Na verdade, essa substituição do Pandas pelo dbt para promover dados da camada Silver para a Gold é considerada uma das melhores práticas absolutas de Engenharia de Dados na Arquitetura Medallion.

A camada **Silver** já resolveu o maior problema do pipeline: ela limpou, estruturou e filtrou os dados brutos da *Bronze/Raw*. Como os campos necessários já estão lá, a transição para a **Gold** envolve regras de negócio, agregações e modelagem dimensional (como tabelas Fato e Dimensão) — tarefas onde o dbt brilha e é infinitamente superior ao Pandas.

Aqui está o porquê de você fazer essa troca agora mesmo e como estruturar isso:

🚀 Por que usar dbt (em vez de Pandas) entre Silver e Gold?

1. Linhagem de Dados Automática (Data Lineage)

Na camada Gold, você costuma cruzar várias tabelas Silver para gerar uma visão de negócio (ex: juntar `silver_clientes` + `silver_vendas` ➡️ `gold_faturamento_mensal`).

- **Com dbt:** Ele entende essas conexões automaticamente através da função `{{ ref() }}` e gera um gráfico visual de dependências (DAG). Se a tabela de vendas mudar, o dbt sabe exatamente quais tabelas Gold precisam ser atualizadas.
- **Com Pandas:** Você teria que gerenciar manualmente a ordem de execução de múltiplos scripts `.py` ou notebooks.

2. Governança e Qualidade (Data Quality)

A camada Gold alimenta os relatórios de diretores e ferramentas de BI, então o dado precisa estar impecável.

- **Com dbt:** Você pode aplicar testes nativos diretamente no arquivo de configuração (`.yml`), garantindo de forma automatizada que a chave primária da sua tabela Gold é única (`unique`) e não possui valores nulos (`not_null`) a cada execução do pipeline.

3. Idempotência e Desempenho (Incremental Models)

Lembra do conceito de idempotência?

- **Com dbt:** Se a sua camada Silver receber milhões de novas linhas por dia, você pode configurar o modelo dbt da Gold como `incremental`. O dbt vai processar de forma inteligente **apenas as linhas novas**, inserindo ou atualizando (*upsert*) o destino sem que você precise reescrever lógica complexa de banco de dados.
- **Com Pandas:** Você teria que ler toda a base Gold existente na memória, concatenar com os dados novos do Pandas, remover duplicadas e sobrescrever o arquivo inteiro. Isso consome muita memória RAM e tempo.

🛠️ Como fica a arquitetura do seu dbt nesse cenário?

No seu projeto dbt, você estruturará a pasta `models/` espelhando a sua arquitetura medallion:

text

```
None
models/
├── silver/
│   ├── stg_clientes.sql      <-- Lê da camada Raw/Bronze
│   └── stg_vendas.sql
└── gold/
    ├── dim_clientes.sql      <-- Usa {{ ref('stg_clientes') }}
    └── fct_vendas.sql        <-- Une as silvers em modelo
Fato/Dimensão
```

Use o código com cuidado.

Se o seu armazenamento físico da camada Silver forem arquivos `.parquet` salvos em um Data Lake (como AWS S3), você pode usar o dbt conectado ao **Databricks**, **BigQuery**, ou de forma local e open-source com o **dbt-duckdb**, que consegue ler esses arquivos `.parquet` diretamente por comandos SQL performáticos e salvar o resultado final (Gold) de volta em formato tabular ou `.parquet`.

📌 Resumo da recomendação

Deixe o Pandas (ou ferramentas de extração) focado apenas em coletar e despejar o dado bruto na camada inicial. **Assim que o dado virar tabular e estruturado (Silver), o dbt deve assumir o controle total até a entrega final na camada Gold.**

Qual tecnologia você está usando atualmente para armazenar esses arquivos da camada Silver? É um **banco de dados** (como PostgreSQL/Snowflake), um **Data Lake em nuvem** (S3/Azure) ou **arquivos locais**? Posso te mostrar um exemplo de como ficaria a sintaxe de um modelo dbt Gold puxando dados de tabelas Silver.

e como e formato ele envia depois de transformado em silver para gold?

Quando o **dbt** assume o processo de transformação para promover os dados da camada **Silver** para a **Gold**, o formato em que ele envia e salva os dados depende do "banco de dados" (adapter) que você configurou para ele gerenciar.

O dbt não cria um arquivo próprio. Ele simplesmente diz ao motor de processamento como o dado deve ser salvo. Na arquitetura Medallion moderna, os formatos mais comuns para salvar os dados transformados na camada Gold são:

1. Se a sua arquitetura é baseada em Data Lake / Lakehouse (Databricks, AWS S3, local com DuckDB)

Se você trabalha com arquivos, o dbt instruirá o motor a salvar a camada Gold em formatos de tabelas analíticas abertas. Os principais são:

- **Delta Lake (`.delta`):** É o padrão da indústria (muito usado no Databricks). Ele armazena os dados em arquivos Parquet por baixo dos panos, mas adiciona uma camada de transações ACID (garantindo que se o dbt falhar no meio da gravação da Gold, o relatório anterior não seja corrompido).
- **Parquet (`.parquet`):** Se você estiver usando o `dbt-duckdb` localmente ou em uma arquitetura de nuvem mais simples, o dbt salvará o resultado final diretamente como um ou mais arquivos `.parquet` altamente compactados e otimizados para leitura em ferramentas de BI (como Power BI ou Metabase).
- **Apache Iceberg:** Outro formato moderno de tabela aberta muito comum para quem usa dbt com Snowflake, AWS Athena ou BigQuery apontando para arquivos em buckets.

2. Se a sua arquitetura é baseada em um Cloud Data Warehouse (Snowflake, BigQuery, Redshift)

Se o seu destino Gold é um banco de dados analítico na nuvem, o dbt salva os dados no formato nativo daquele banco. No entanto, o dbt permite que você escolha a **estratégia de materialização** através das configurações do seu arquivo SQL. Os formatos lógicos de envio e salvamento no banco são:

- **Tabela Nativa (`table`):** O dbt envia um comando que reconstrói a tabela Gold do zero a cada execução (ótimo para bases menores).

- **Incremental (`incremental`):** O dbt envia apenas as linhas que mudaram ou são novas na Silver. Ele faz um `UPSERT` ou `MERGE` diretamente nas tabelas da Gold.
- **View (`view`):** O dbt cria apenas um "atalho" SQL. O dado continua fisicamente na Silver, mas a Gold expõe a transformação em tempo real quando alguém consulta.

🧱 Como isso é definido no código dbt?

Dentro do próprio arquivo `.sql` da sua camada Gold, você define como quer que o dbt salve esse formato final usando o bloco `config`.

Veja este exemplo usando **Databricks/DuckDB** salvando em formato **Delta** (ou Parquet) de maneira **incremental** (idempotente):

sql

```
None
{{ config(
    materialized='incremental',
    file_format='delta',            -- Pode ser 'parquet',
'iceberg', etc.
    unique_key='id_venda'           -- Chave para evitar duplicados
na Gold
) }}

SELECT
    vendas.id_venda,
    clientes.nome_cliente,
    vendas.valor_total,
    vendas.data_venda
FROM {{ ref('silver_vendas') }} AS vendas
LEFT JOIN {{ ref('silver_clientes') }} AS clientes
    ON vendas.id_cliente = clientes.id_cliente

{% if is_incremental() %}
    -- Se o dbt já rodou antes, ele só envia o que mudou na Silver
desde a última carga
```

```
    WHERE vendas.data_atualizacao >= (SELECT MAX(data_venda) FROM {{
this }})
{% endif %}
```

Use o código com cuidado.

📌 Resumo visual do fluxo de formatos

text

```
None
[Silver] (.parquet / .delta)
       ⬇️
[dbt lê via SQL] (O dbt processa a lógica de negócio na memória do
banco/motor)
       ⬇️
[Gold] Envia o comando de salvamento ➡️ Salva como (.delta, .parquet
ou Tabela Nativa do Data Warehouse)
```

Use o código com cuidado.

O grande ganho aqui é que você não precisa fazer como no Pandas (`df.to_parquet('caminho/gold.parquet')`), gerenciando partições e caminhos manualmente. O dbt sabe onde a Gold deve morar com base no seu arquivo de conexões (`profiles.yml`).

Qual é o **orquestrador ou tecnologia** que você está usando (ou planeja usar) para rodar esse pipeline? (Ex: **Airflow**, **Prefect**, ou rodando scripts direto no **Github Actions/terminal**?) Sabendo disso, posso te explicar como conectar a execução do dbt ao seu fluxo atual.

> meu data warehouse que defini ao final que recebe os dados de gold é duckdb.

Excelente escolha! O DuckDB é considerado o "Pandas com esteroides" para engenharia de dados moderna. Ele é absurdamente rápido para ler e escrever

arquivos locais ou na nuvem (como AWS S3) e gerencia a memória de forma impecável.

Quando o seu destino final é o DuckDB, o formato e a forma como o dbt envia os dados da camada **Silver** para a **Gold** tornam-se muito simples e eficientes.

💾 Como os dados da Gold são salvos fisicamente no DuckDB?

Usando o plugin `dbt-duckdb`, você tem duas opções principais para o formato final da sua camada Gold:

Opção A: Dentro do arquivo de banco de dados do DuckDB (`.duckdb` ou `.db`)

Por padrão, se você não mudar nada nas configurações, o dbt enviará os dados transformados para dentro do próprio arquivo físico do banco DuckDB.

- **Como funciona:** O dbt executa o SQL e armazena o resultado em formato tabular nativo do DuckDB (que já é colunar, altamente compactado e otimizado).
- **Ideal para:** Quando o seu consumo posterior (a ferramenta de BI ou o dashboard) consegue ler diretamente o arquivo `.duckdb`.

Opção B: Como arquivos externos `.parquet` (Mais recomendada para arquiteturas abertas)

Você pode configurar o `dbt-duckdb` para que, ao processar a camada Gold, ele **escreva o resultado diretamente em um arquivo `.parquet` externo** no seu disco ou em um servidor de arquivos, mantendo apenas a referência (o atalho) dentro do DuckDB.

- **Como funciona:** O dado final da Gold vira um arquivo físico `.parquet`.
- **Ideal para:** Se você precisar que outras ferramentas que não conhecem o DuckDB (como o Power BI local, outro script em Python ou uma ferramenta de nuvem) leiam os dados da camada Gold de forma totalmente independente.

🛠️ Como configurar isso no seu dbt?

Para que o dbt salve os dados transformados da Gold no formato correto, você configura o arquivo `dbt_project.yml` ou o topo do seu arquivo `.sql` da Gold.

Veja como configurar a camada Gold para exportar automaticamente em **`.parquet`** usando o DuckDB:

sql

```
None
-- models/gold/fct_vendas.sql

{{ config(
    materialized='table',
    plugin='parquet',
    external_location="path/to/data/gold/fct_vendas.parquet"
) }}

SELECT
    vendas.id_venda,
    clientes.nome_cliente,
    vendas.valor_total,
    vendas.data_venda
FROM {{ ref('stg_vendas') }} AS vendas  -- Puxando da Silver
LEFT JOIN {{ ref('stg_clientes') }} AS clientes
    ON vendas.id_cliente = clientes.id_cliente
```

Use o código com cuidado.

O que acontece por trás dos panos quando você roda **`dbt run`**?

1. O dbt abre uma conexão com o DuckDB.
2. O DuckDB lê as tabelas ou arquivos da camada **Silver** em alta velocidade (usando processamento multi-core e sem estourar a memória RAM).
3. O DuckDB faz o `JOIN` e as agregações da Gold.

4. O DuckDB gera e grava um arquivo `fct_vendas.parquet` novinho e otimizado no caminho definido na `external_location`.

✨ A grande vantagem sobre o Pandas

Diferente do Pandas, onde você precisa escrever códigos complexos e demorados para ler múltiplos Parquets, juntá-los e salvá-los, no DuckDB + dbt você escreve apenas **SQL puro**, e o motor do DuckDB garante que a escrita do arquivo `.parquet` final da Gold seja feita utilizando compressão máxima (Snappy) e ordenação ideal para consultas rápidas.

Como você pretende consumir esses dados da camada Gold ao final do pipeline? Você vai conectar o arquivo do **DuckDB em uma ferramenta de BI** (como o Microsoft Power BI ou o Evidence), ou vai ler esses dados usando outra aplicação? Dependendo do consumo, posso te dar uma dica de como estruturar os caminhos dos arquivos.

> hoje meus dados de silver vão para o gold como parquet. Posso processar no silver como dbt e depois desse processamento ele envia para o duckdb e envia para o gold como .parquet?

Sim, você pode fazer exatamente isso. Com o dbt e o DuckDB trabalhando juntos, todo esse fluxo de processamento e exportação acontece de forma nativa, automática e extremamente rápida.

Você não precisa de nenhuma etapa manual para extrair os dados e salvá-los de novo em `.parquet`. O próprio **DuckDB funciona como o motor que processa o dbt** e, ao mesmo tempo, gera os arquivos finais da camada Gold.

🔄 Como o Fluxo Funciona na Prática

Quando você executa o comando `dbt run`, o processo acontece em um único fluxo integrado:

```
None
[Camada Silver] (.parquet)
       ⬇️ (O dbt lê usando o motor do DuckDB)
[DuckDB] Processa os SQLs da Gold (Joins, Regras de Negócio,
Agregações)
       ⬇️ (O dbt instrui o DuckDB a exportar o resultado
diretamente)
[Camada Gold] (.parquet atualizado no seu disco ou nuvem)
```

🛠️ Como estruturar isso no seu código dbt

Para que o dbt processe no DuckDB e salve a Gold diretamente como `.parquet`, você usa o recurso de **Tabelas Externas (`external_location`)** do plugin `dbt-duckdb`.

Basta configurar o topo do seu arquivo SQL da camada Gold (por exemplo, dentro de `models/gold/fct_vendas.sql`):

sql

```
None
{{ config(
    materialized='table',
    plugin='parquet',

external_location='caminho/para/sua/pasta/gold/fct_vendas.parquet'
) }}

-- Seu código SQL de transformação Gold aqui:
SELECT
    v.id_venda,
    c.nome_cliente,
    v.valor_total,
    v.data_venda
FROM {{ ref('silver_vendas') }} v  -- O dbt lê automaticamente o
.parquet da Silver
```

```
LEFT JOIN {{ ref('silver_clientes') }} c
    ON v.id_cliente = c.id_cliente
```

Use o código com cuidado.

💎 As vantagens dessa abordagem para o seu cenário:

1. **Adeus caminhos manuais (`paths`):** No seu código SQL, você não escreve caminhos de arquivos como `pandas.read_parquet('path/silver...')`. Você apenas usa a função `{{ ref('nome_da_tabela_silver') }}`. O dbt e o DuckDB descobrem os caminhos sozinhos com base nas suas configurações.
2. **Performance brutal:** O DuckDB lê os `.parquet` da Silver, processa as transformações na memória de forma altamente otimizada (usando vetores) e já cospe o `.parquet` da Gold de forma direta e compactada.
3. **Linhagem e Governança:** O dbt vai gerar o gráfico visual mostrando exatamente quais arquivos `.parquet` da Silver deram origem aos arquivos `.parquet` da Gold.

Para que tudo funcione perfeitamente, certifique-se apenas de que no seu arquivo `profiles.yml` (onde fica a conexão do dbt), o seu schema da camada `silver` também esteja configurado apontando para onde os arquivos `.parquet` da Silver estão salvos.

Os seus arquivos `.parquet` da camada Silver e Gold ficam salvos na sua **máquina local** (ou servidor) ou você utiliza algum **Cloud Storage** (como AWS S3 ou Google Cloud Storage)? Posso te mostrar como ajustar o seu arquivo `profiles.yml` para que o DuckDB gerencie esses caminhos de arquivos perfeitamente.